

Automatización del testing

Clase 3:Automatización de Test | JUnit Framework| setUp| TearDown|Test negativos

Valeria Bengolea

Estas transparencias están basadas en las desarrolladas por Ammann & Offutt como acompañamiento de su libro Introduction to Software Testing (2nd Edition) y por Manuel Núñez

Testing “ad hoc” (no sistemático)

Todo programador está habituado al testing.

En muchos casos, la forma en la que hacemos testing es “ad hoc”, es decir, no sistemática.

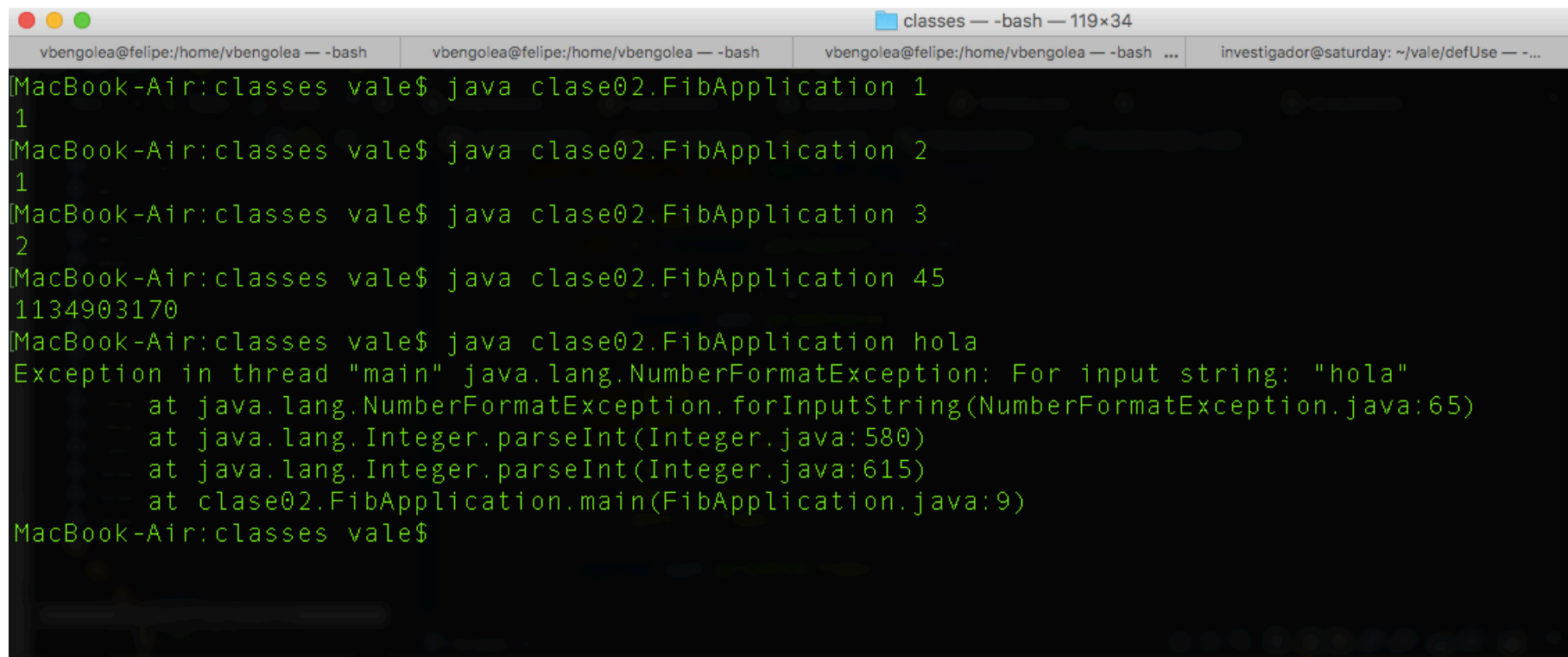
Testing “ad hoc” (no sistemático)

Ej: supongamos que queremos testear la siguiente función:

```
public class FibApplication {  
  
    public static void main(String[] args) {  
  
        long fibn=0;  
        int n =Integer.parseInt(args[0]);  
  
        if (n == 0 || n == 1)  
            System.out.println(n);  
  
        long ant2 = 0;  
        long ant1 = 1;  
        for (int i=2; i<=n;i++){  
            fibn = ant1 + ant2;  
            ant2 = ant1;  
            ant1 = fibn;  
        }  
        System.out.println(fibn);  
    }  
}
```

Testing “ad hoc” (cont.)

En este caso, podemos **testear** la aplicación **directamente** desde la línea de comandos:

A screenshot of a macOS terminal window titled 'classes — -bash — 119x34'. The terminal shows a series of commands to run a Java application named 'FibApplication'. The first three commands pass integer arguments (1, 2, 3) and return integer outputs (1, 1, 2). The fourth command passes '45' and returns '1134903170'. The fifth command passes the string 'hola', which results in a 'NumberFormatException' exception being thrown, with the message 'Exception in thread "main" java.lang.NumberFormatException: For input string: "hola"'. The stack trace shows the exception originates from the 'main' method of 'FibApplication.java' at line 9, which calls 'Integer.parseInt' at line 615, which in turn calls 'forInputString' at line 580 of 'Integer.java' and 'forInputString' at line 65 of 'NumberFormatException.java'.

```
MacBook-Air:classes vale$ java clase02.FibApplication 1
1
MacBook-Air:classes vale$ java clase02.FibApplication 2
1
MacBook-Air:classes vale$ java clase02.FibApplication 3
2
MacBook-Air:classes vale$ java clase02.FibApplication 45
1134903170
MacBook-Air:classes vale$ java clase02.FibApplication hola
Exception in thread "main" java.lang.NumberFormatException: For input string: "hola"
    at java.lang.NumberFormatException.forInputString(NumberFormatException.java:65)
    at java.lang.Integer.parseInt(Integer.java:580)
    at java.lang.Integer.parseInt(Integer.java:615)
    at clase02.FibApplication.main(FibApplication.java:9)
MacBook-Air:classes vale$
```

Testing “ad hoc” (cont.)

Ej: En caso que lo que tengamos no sea una aplicación, sino una **funcionalidad interna**:

```
public class SimpleRoutines{
    public static long fibonacciEfi(long n){
        long fibn=0;
        if (n == 0 || n == 1)
            return n;

        long ant2 = 0;
        long ant1 = 1;
        for (int i=2; i<=n;i++){
            fibn = ant1 + ant2;
            ant2 = ant1;
            ant1 = fibn;
        }
        return fibn;
    }
}
```

El testing “ad hoc” se vuelve **más difícil**: tenemos que programar un **arnés para la rutina** (i.e., un “main” que la invoque).

Dificultades del Testing Ad Hoc

- ★ No almacena los tests para pruebas futuras.
- ★ Requiere la construcción manual de “arneses” para la prueba de funcionalidades “internas” (no directamente invocables desde la interfaz de la aplicación).
- ★ Requiere la inspección humana de la salida de los tests: se decide manualmente si el test pasó o no.

¿En qué consiste la automatización de los tests?

Es el uso de software para controlar la ejecución de tests, comparar los resultados obtenidos con los esperados, setear las precondiciones del test y otras tareas

La automatización del proceso de testing permite reducir:

Costos

Errores humanos.

Además, reduce sensiblemente el costo asociado con regression testing.

¿Qué es *software testability*?

El grado en el que un sistema o componente facilita el establecimiento de criterios de testing y la ejecución de los tests para determinar si dichos criterios se cumplen.

La testeabilidad depende, fundamentalmente, de dos problemas:

¿Qué es *software testability*?

El grado en el que un sistema o componente facilita el establecimiento de criterios de testing y la ejecución de los tests para determinar si dichos criterios se cumplen.

La testeabilidad depende, fundamentalmente, de dos problemas:

★ Cómo se proporcionan valores de los tests al software.

¿Qué es *software testability*?

El grado en el que un sistema o componente facilita el establecimiento de criterios de testing y la ejecución de los tests para determinar si dichos criterios se cumplen.

La testeabilidad depende, fundamentalmente, de dos problemas:

- ★ Cómo se proporcionan valores de los tests al software.
- ★ Cómo se **observan los resultados** de la ejecución de tests.

Observabilidad & controlabilidad

Cuán fácil es observar el comportamiento del programa en términos de sus outputs, efectos en el entorno, y otras componentes software y hardware.

El software que afecta a dispositivos hardware, bases de datos y ficheros remotos suele tener baja observabilidad.

El software que recibe inputs a través de teclado (o similar) es fácil de controlar.

Por el contrario, inputs destinados a sensores o sistemas distribuidos suelen conllevar problemas.

Componentes de un test

En un test podemos distinguir estos componentes principales:

Componentes de un test

En un test podemos distinguir estos componentes principales:

Los **valores de los inputs** que se necesitan para completar la ejecución del software bajo prueba. Inputs usadas para satisfacer los requisitos de tests

Componentes de un test

En un test podemos distinguir estos componentes principales:

Los **valores de los inputs** que se necesitan para completar la ejecución del software bajo prueba. Inputs usadas para satisfacer los requisitos de tests

El **resultado** que producirá el test si el software bajo prueba se comporta de la forma esperada.

Componentes de un test

En un test podemos distinguir estos componentes principales:

Los **valores de los inputs** que se necesitan para completar la ejecución del software bajo prueba. Inputs usadas para satisfacer los requisitos de tests

El **resultado** que producirá el test si el software bajo prueba se comporta de la forma esperada.

Un **oráculos** que decide si el test ha **pasado** o **fallado**.

Componentes de un test

En algunas situaciones necesitamos proveer otros inputs para obtener observabilidad/controlabilidad

★ **Valores prefijos**. Inputs necesarios para llevar el software al estado apropiado para recibir los inputs del test.

Componentes de un test

En algunas situaciones necesitamos proveer otros inputs para obtener observabilidad/controlabilidad

★ **Valores prefijos**. Inputs necesarios para llevar el software al estado apropiado para recibir los inputs del test.

★ **Valores postfijos**. Inputs que debemos mandar al software después de que se envíen los valores de los tests. Distinguimos dos tipos:

Componentes de un test

En algunas situaciones necesitamos proveer otros inputs para obtener observabilidad/controlabilidad

★ **Valores prefijos**. Inputs necesarios para llevar el software al estado apropiado para recibir los inputs del test.

★ **Valores postfijos**. Inputs que debemos mandar al software después de que se envíen los valores de los tests. Distinguimos dos tipos:

★ **Valores de verificación**. Valores necesarios para ver el resultado de la aplicación del test.

Componentes de un test

En algunas situaciones necesitamos proveer otros inputs para obtener observabilidad/controlabilidad

★ **Valores prefijos**. Inputs necesarios para llevar el software al estado apropiado para recibir los inputs del test.

★ **Valores postfijos**. Inputs que debemos mandar al software después de que se envíen los valores de los tests. Distinguimos dos tipos:

★ **Valores de verificación**. Valores necesarios para ver el resultado de la aplicación del test.

★ **Valores de salida**. Valores o comandos necesarios para terminar el programa o volver a un estado estable.

Poniendo todo junto...

Un **test** consiste en el **valor de sus inputs**, los **valores de prefijo**, los **valores de postfijo**, y resultados esperados para una ejecución y evaluación completa del SUT.

Poniendo todo junto...

Un **test** consiste en el **valor de sus inputs**, los **valores de prefijo**, los **valores de postfijo**, y resultados esperados para una ejecución y evaluación completa del SUT.

Si queremos automatizar el proceso de testing, necesitamos **scripts ejecutables**.

Poniendo todo junto...

Un **test** consiste en el **valor de sus inputs**, los **valores de prefijo**, los **valores de postfijo**, y resultados esperados para una ejecución y evaluación completa del SUT.

Si queremos automatizar el proceso de testing, necesitamos **scripts ejecutables**.

Un **script de testing ejecutable** es un **test** que está preparado para ser **ejecutado automáticamente** sobre el software y generar un **informe**.

Qué es JUnit?

JUnit: es framework de apoyo al testing para Java:

- ★ Define la estructura standard de un test.
- ★ Permite almacenar los tests como scripts ejecutables
- ★ Permite organizar tests en suites.
- ★ Provee aserciones para chequear automáticamente los resultados esperados.
- ★ Ofrece entornos para la ejecución de tests y suites.
- ★ Reporta información detallada sobre las pruebas, en especial sobre las pruebas que fallan.

Qué es JUnit?

JUnit puede ser usado para testar

- ★ Métodos individuales (testing unitario)
- ★ Métodos de un mismo módulo. (Testing de módulo)
- ★ La interacción entre módulos (Testing de Integración)

Está destinado principalmente a pruebas unitarias y de integración, no a pruebas de sistemas.



Para todos los lenguajes modernos existe algun framework con características similares a JUnit (ej Python -> Pytest)

JUnit es software libre (junit.org)

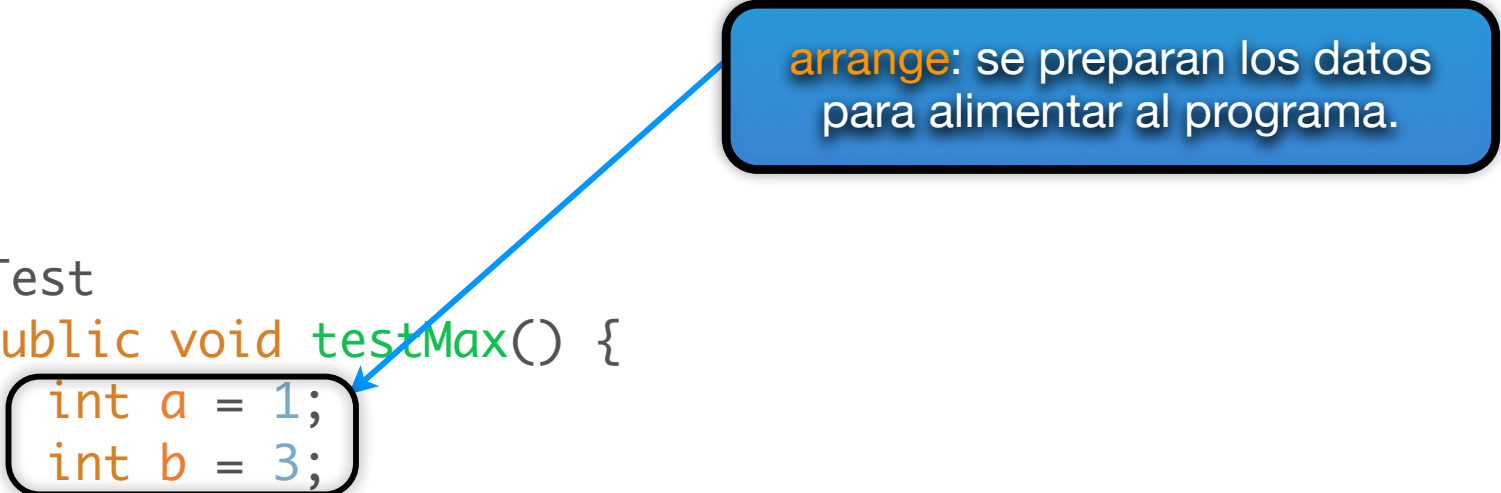
Estructura de un test

```
@Test
public void testMax() {
    int a = 1;
    int b = 3;
    int res = SimpleRoutines.max(a,b);
    assertTrue(res == 3);
}
```

Estructura de un test

arrange: se preparan los datos para alimentar al programa.

```
@Test
public void testMax() {
    int a = 1;
    int b = 3;
    int res = SimpleRoutines.max(a,b);
    assertTrue(res == 3);
}
```



Estructura de un test

```
@Test
public void testMax() {
    int a = 1;
    int b = 3;
    int res = SimpleRoutines.max(a,b);
    assertTrue(res == 3);
}
```

arrange: se preparan los datos para alimentar al programa.

act: se ejecuta el programa con los datos contruidos.

Estructura de un test

```
@Test
public void testMax() {
    int a = 1;
    int b = 3;
    int res = SimpleRoutines.max(a,b);
    assertTrue(res == 3);
}
```

arrange: se preparan los datos para alimentar al programa.

act: se ejecuta el programa con los datos contruidos.

assert: se evalúa si los resultados obtenidos se corresponden con lo esperado.

Algunas Aserciones JUnit

assertTrue(<expr>): verifica que <expr> evalúe a true.

assertFalse(<expr>): verifica que <expr> evalúe a false.

assertEquals(<expr1>, <expr2>): verifica que <expr1> y <expr2> evalúen al mismo valor.

assertArrayEquals(<array1>, <array2>): verifica que <array1> y <array2> sean iguales, elemento a elemento.

assertNotNull(<object>): verifica que <object> no sea null.

assertNull(<object>): verifica que <object> sea null.

assertNotSame(<object1>, <object2>): verifica que <object1> y <object2> no sean el mismo objeto.

Algunas aserciones JUnit

Expresiones lambda en aserciones

```
@Test
void groupedAssertions() {
    // In a grouped assertion all assertions are executed, and all
    // failures will be reported together.
    assertAll("person",
        () -> assertEquals("Jane", person.getFirstName()),
        () -> assertEquals("Doe", person.getLastName())
    );
}
```

```
@Test
void dependentAssertions() {
    // Within a code block, if an assertion fails the
    // subsequent code in the same block will be skipped.
    assertAll("properties",
        () -> {
            String firstName = person.getFirstName();
            assertNotNull(firstName);

            // Executed only if the previous assertion is valid.
            assertAll("first name",
                () -> assertTrue(firstName.startsWith("J")),
                () -> assertTrue(firstName.endsWith("e"))
            );
        },
        () -> { . . . }
    );
}
```

Matchers hamcrest y assertThat

Las versiones más recientes de JUnit proveen una nueva forma de escribir aserciones para los test

Único método para describir resultados esperados:

`assertThat(<object>, <matcher>)`

Un “matcher” hamcrest es una descripción declarativa de una característica deseable de un objeto. Por ejemplo:

`assertThat(1, hasItems(2,3));`

`assertThat(list, everyItem(greaterThan(1)));`

Matchers hamcrest

Algunos matchers:

Core: anything, describedAs, is

Logical: allOf, anyOf, not

Object: equalTo, instanceOf, nullValue, sameInstance

Collections: hasItem, hasItemInArray, contains, containsInAnyOrder

Numbers: greaterThan, greaterThanOrEqualTo, lessThan, lessThanOrEqualTo

Text: equalToIgnoringCase, equalToIgnoringWhiteSpace, containsString, endsWith, startsWith

<http://hamcrest.org/JavaHamcrest/javadoc/1.3/org/hamcrest/CoreMatchers.html>

<http://hamcrest.org/JavaHamcrest/javadoc/1.3/org/hamcrest/Matchers.html>

Matchers hamcrest: Ejemplos

El String str es igual a “b” o a “c”:

```
assertThat(str, anyOf(equalTo("b"), equalTo("c")));
```

“HoLa” es igual a “hola” si se ignoran mayúsculas:

```
assertThat("HoLa", equalToIgnoringCase("hola"));
```

El objeto obj no es null:

```
assertThat(obj, not(isNull));
```

El objeto obj es de tipo Student (o cualquier subtipo de Student):

```
assertThat(obj, instanceOf(Student.class));
```

El arreglo arr contiene el elemento 5:

```
assertThat(arr, hasItemInArray(5));
```

Otra librería para escribir aserciones

AssertJ:

<https://assertj.github.io/doc/#assertj-overview>

Características deseables de los tests

- ★ Los tests deben ser:
- ★ Automáticos
 - * Fácilmente ejecutables por cualquier persona involucrada en el proyecto
 - * Ejecutables con sólo “presionar un botón”
- ★ Repetibles (con resultados consistentes)
- ★ Fáciles de implementar y comprender
- ★ Independientes entre si
- ★ Rápidos y eficientes en el uso de recursos

Timeouts

Algunos tests ejecutan funcionalidades que podrían no terminar, o demandar mucho tiempo.

Testeando
implementación
eficiente de fibonacci

```
@Test
@Timeout(value = 1000, unit = TimeUnit.MILLISECONDS)
public void test1() {
    long res = Fibonacci.fibonacciEfi(45);
    assertEquals(res, 1134903170);
}

@Test
void timeoutNotExceeded()
{
    assertTimeout(ofMillis(1000), () -> {
        long res = Fibonacci.fibonacciEfi(45);
    });
}
```

Suites de tests

Cuando el SUT está compuesto por varias clases, o simplemente varios métodos de una clase, resulta claro que debemos organizar los tests.

Los tests se organizan en **test suites**.

una **test suite** es simplemente **un conjunto de tests**.

Suites de tests

En JUnit, la organización es un poco más elaborada:

un Test Case es un conjunto de tests

una Test Suite es una colección de test cases:

```
@RunWith(Suite.class)
@Suite.SuiteClasses({
    UnitTests1.class,
    UnitTests2.class,
    ModuleTests.class,
    IntegrationTests.class
})
class AllUnitTest {}
```

JUnit4

```
@SelectPackages("mypackage")
@ExcludePackages("otherPackage")
....
```

```
@Suite
@SelectClasses({
    UnitTests1.class,
    UnitTests2.class,
    ModuleTests.class,
    IntegrationTests.class
})
class AllUnitTest {}
```

JUnit5

Fixtures de Test en JUnit

En una suite puede haber tests que comparten datos

El **Fixture** define el estado de un test

- ★ Útil cuando objetos y variables son usados por más de un test
- ★ Usado para controlar *initializations* (valores prefijos)
- ★ Usado para controlar *reset values* (valores postfijos)

Diferentes test pueden usar los objetos sin compartir el estado

Objetos usados en fixtures de test deberían ser declarados como atributos de clase

Objetos deben inicializarse en un método etiquetado con **@BeforeEach**

Deben restaurarse o eliminarse en un método etiquetado con **@AfterEach**

De este modo, diferentes tests pueden usar los objetos sin compartir el estado

Suites de Test y Datos Compartidos: Un Ejemplo

```
public class Minefield {  
    private Mine[][] field;  
  
    /**  
     * @param (x,y) a valid position on mineField  
     * @return the number of neighbors position of (x,y) with mines  
     */  
    public int minedNeighbours(int x, int y){  
        ...  
    }  
  
    ...  
}  
  
public class Mine {  
    private boolean isMined;  
    private boolean isMarked;  
    private boolean isOpened;  
    ...  
}
```


Suites de Test y Datos Compartidos: Un Ejemplo

```
public class Minefield {  
  
    private Mine[][] field;  
  
    /**  
     * @param (x,y) a valid position on mineField  
     * @return the number of neighbors position of (x,y) with mines  
     */  
    public int minedNeighbours(int x, int y){  
        ...  
    }  
  
    ...  
}  
  
public class Mine {  
    private boolean isMined;  
    private boolean isMarked;  
    private boolean isOpened;  
    ...  
}
```

- Para poder testear minedNeighbours, debemos crear el minefield, y ubicar minas en lugares específicos.

Suites de Test y Datos Compartidos: Un Ejemplo

```
@Test
public void testMinedNeighbours2() {
    Minefield testingField = new Minefield(8,8);
    testingField.putMine(0, 0);
    testingField.putMine(3, 4);
    testingField.putMine(4, 3);
    testingField.putMine(2, 2);
    testingField.putMine(0, 7);
    testingField.putMine(7, 7);
    testingField.putMine(5, 1);
    testingField.putMine(4, 7);

    int number = testingField.minedNeighbours(3, 3);
    org.junit.Assert.assertTrue(number == 3);
}
```

	0	1	2	3	4	5	6	7
0	1							
1						1		
2			1					
3					1			
4				1				
5								
6								
7	1				1			1

```
@Test
public void testMinedNeighbours1() {
    Minefield testingField = new Minefield(8,8);
    testingField.putMine(0, 0);
    testingField.putMine(3, 4);
    testingField.putMine(4, 3);
    testingField.putMine(2, 2);
    testingField.putMine(0, 7);
    testingField.putMine(7, 7);
    testingField.putMine(5, 1);
    testingField.putMine(4, 7);
    int number = testingField.minedNeighbours(1, 0);
    org.junit.Assert.assertTrue(number == 1);
}
```

Suites de Test y Datos Compartidos: Un Ejemplo

El proceso `setUp` crea el escenario adecuado para la ejecución de tests de `minedNeighbours`.

Se **ejecuta antes** de cada test.

```
@BeforeEach
public void setUp() throws Exception {
    if (testingField == null) {
        testingField = new Minefield(8,8);
    }
    for (int i=0; i<8; i++) {
        for (int j=0; j<8; j++) {
            testingField.removeMine(i, j);
            testingField.unmark(i, j);
            testingField.close(i, j);
        }
    }
    testingField.putMine(0, 0);
    testingField.putMine(3, 4);
    testingField.putMine(4, 3);
    testingField.putMine(2, 2);
    testingField.putMine(0, 7);
    testingField.putMine(7, 7);
    testingField.putMine(5, 1);
    testingField.putMine(4, 7);
}
```

	0	1	2	3	4	5	6	7
0	1							
1						1		
2			1					
3					1			
4				1				
5								
6								
7	1				1			1

```
@Test
public void testMinedNeighbours1() {
    int number = testingField.minedNeighbours(1, 0);
    org.junit.Assert.assertTrue(number == 1);
}

@Test
public void testMinedNeighbours2() {
    int number = testingField.minedNeighbours(3, 3);
    org.junit.Assert.assertTrue(number == 3);
}
```

Suites de Test y Datos

Compartidos: Un Ejemplo

El proceso `setUp` crea el escenario adecuado para la ejecución de tests de `minedNeighbours`.

Se **ejecuta antes** de cada test.

```
@BeforeAll
@BeforeEach
public void setUp() throws Exception {
    if (testingField == null) {
        testingField = new Minefield(8,8);
    }
    for (int i=0; i<8; i++) {
        for (int j=0; j<8; j++) {
            testingField.removeMine(i, j);
            testingField.unmark(i, j);
            testingField.close(i, j);
        }
    }
    testingField.putMine(0, 0);
    testingField.putMine(3, 4);
    testingField.putMine(4, 3);
    testingField.putMine(2, 2);
    testingField.putMine(0, 7);
    testingField.putMine(7, 7);
    testingField.putMine(5, 1);
    testingField.putMine(4, 7);
}
```

	0	1	2	3	4	5	6	7
0	1							
1						1		
2			1					
3					1			
4				1				
5								
6								
7	1				1			1

```
@Test
public void testMinedNeighbours1() {
    int number = testingField.minedNeighbours(1, 0);
    org.junit.Assert.assertTrue(number == 1);
}

@Test
public void testMinedNeighbours2() {
    int number = testingField.minedNeighbours(3, 3);
    org.junit.Assert.assertTrue(number == 3);
}
```

Sobre setUp y tearDown

- ★ Es importante que no haya dependencias entre tests diferentes.
 - ★ No hay ninguna garantía sobre el orden en el que se ejecutan los tests
- ★ Las rutinas setUp (@BeforeEach) y tearDown (@AfterEach) ayudan a garantizar la ausencia de dependencias entre diferentes tests.
 - ★ El método anotado con @BeforeEach se ejecuta antes de cada test de la suite.
 - ★ El método anotado con @AfterEach se ejecuta después de cada test de la suite,
 - ★ aún si el test lanza excepciones.

Fixtures globales

A veces, setup y teardown realizan operaciones costosas

O ejecutarlos antes y después de cada test no es la mejor opción

@BeforeAll se invoca (una vez) antes de ejecutar los tests de la clase (inicializar datos/recursos globales a los tests de una misma clase)

@AfterAll se invoca (una vez) después de ejecutar todos los tests de la clase (destruir datos/recursos globales a los tests de una misma clase).

```
@BeforeAll
public static void openConnection(){
    //setup
    connection = DB.open(...)
}
@AfterAll
public static void closeConnection(){
    // teardown
    connection.close()
}
...
```

Test “negativos” en JUnit

Los **tests negativos** son aquellos en los ejercitamos el software bajo test en circunstancias que violan la “precondición” del software bajo test. En muchos casos, se espera que los tests negativos lancen excepciones.

```
@Test
public void minMaxArrayTest() {
    final int[] a = {1, 2, 3};

    Assertions.assertThrows(IllegalArgumentException.class, () -> {
        IntPair res = SimpleRoutines.minMaxArray(a, 0, 4);
    });
}
```

UpperBound fuera del rango
correcto

```
@Test
public void test1() {
    BoundedQueue q = new BoundedQueue();
    Assertions.assertThrows(IllegalArgumentException.class, () ->
        q.dequeue()
    );
}
```

Desencolar sobre una cola
vacía

Ejemplo

```
public class Min{
    /**
     * Returns the mininum element in a list
     * @param list Comparable list of elements to search
     * @return the minimum element in the list
     * @throws NullPointerException if list is null or
     *         if any list elements are null
     * @throws ClassCastException if list elements are not mutually comparable
     * @throws IllegalArgumentException if list is empty
     */
    public static <T extends Comparable<? super T>> T min (List<? extends T> list){
        if (list.size() == 0){
            throw new IllegalArgumentException ("Min.min");
        }

        Iterator<? extends T> itr = list.iterator();
        T result = itr.next();

        if (result == null) throw new NullPointerException ("Min.min");

        while (itr.hasNext()){
            // throws NPE, CCE as needed
            T comp = itr.next();
            if (comp.compareTo (result) < 0){
                result = comp;
            }
        }
        return result;
    }
}
```


Ejemplo

Test fixture and pre-test
setup method (prefix)



```
public class MinTest{  
    private List<String> list;    // Test fixture  
  
    @BeforeEach // Set up – Called before every test method.  
    public void setUp(){  
        list = new ArrayList<String>();  
    }  
  
    @AfterEach // Tear down – Called after every test method.  
    public void tearDown(){  
        list = null; // redundant in this example!  
    }  
}
```



Post test
teardown
method (postfix)

Min Test Cases: NullPointerException

```
@Test
public void testForNullList(){
    list = null;
    assertThrows(NullPointerException.class, () -> Min.min (list));
}
```

```
@Test
public void testForNullElement(){
    list.add ("cat");
    list.add (null);
    assertThrows(NullPointerException.class, () -> Min.min (list));
}
```

```
@Test
public void testForSoloNullElement(){
    list.add (null);
    assertThrows(NullPointerException.class, () -> Min.min (list));
}
```

Más Test Cases para Min

```
@Test
@SuppressWarnings ("unchecked")
public void testMutuallyIncomparable(){
    List list = new ArrayList();
    list.add ("cat");
    list.add ("dog");
    list.add (1);
    assertThrows(ClassCastException.class, () -> Min.min (list));
}

@Test
public void testEmptyList()
{
    assertThrows(IllegalArgumentException.class, () -> Min.min (list));
}
```

Más Test Cases para Min

```
@Test
@SuppressWarnings ("unchecked")
public void testMutuallyIncomparable(){
    List list = new ArrayList();
    list.add ("cat");
    list.add ("dog");
    list.add (1);
    assertThrows(ClassCastException.class, () -> Min.min (list));
}

@Test
public void testEmptyList()
{
    assertThrows(IllegalArgumentException.class, () -> Min.min (list));
}
```

Special case: Testing for the empty list

El resto de los Test para Min

```
@Test
public void testSingleElement()
{
    list.add ("cat");
    Object obj = Min.min (list);
    assertTrue (obj.equals ("cat"), "Single Element List");
}

@Test
public void testDoubleElement()
{
    list.add ("dog");
    list.add ("cat");
    Object obj = Min.min (list);
    assertTrue (obj.equals ("cat"), "Double Element List");
}
```

El resto de los Test para Min

Finalmente! Un para de test de "Happy Path"

```
@Test
public void testSingleElement()
{
    list.add ("cat");
    Object obj = Min.min (list);
    assertTrue (obj.equals ("cat"), "Single Element List");
}

@Test
public void testDoubleElement()
{
    list.add ("dog");
    list.add ("cat");
    Object obj = Min.min (list);
    assertTrue (obj.equals ("cat"), "Double Element List");
}
```

Resumen: Siete Tests para Min

Five tests with exceptions

1. null list
2. null element with multiple elements
3. null single element
4. incomparable types
5. empty elements

Two without exceptions

6. single element
7. two elements